

Projet ASD : Structures de Données Hybrides

Seydina Mohamed Diop & Abdou Lahate

4 juillet 2026

1 Introduction

Ce Projet porte sur un systeme de gestion de scolarité . En effet on va concevoir un programme qui va permettre a un utilisateur d enregistrer autant d etudiant qu il souhaite accompagner de leur information personnel et pedagogique .Ainsi une serie de message vont s afficher sur le terminal permettant a l utilisateur d echanger avec la machine .Ce dernier fera des series de calculs liés aux **Structures** et **Fonctions** ci dessous et affichera les informations correspondantes.

2 Modélisation des Structures de Données

Voici les Structures de base que nous avons conçues pour notre **Systeme De Scolarité**.

```
1 typedef struct Note
2 {
3     char matier[20];
4     float valeur;
5     int coef;
6     struct Note *note_suivante; // permet de passer au note
        suivant du meme etudiant
7 }Note;
```

2.0.1 Explication de la structure Note

La structure **Note** reçoit et stocke les résultats académiques d'un étudiant pour chaque matière de la filière, ainsi que le coefficient correspondant. Par exemple, pour un étudiant donné, cette structure va enregistrer ses notes au fur et à mesure. Ensuite, grâce au pointeur `struct Note *note_suivante`, la structure se connecte à la note suivante du même étudiant. C'est ce mécanisme

qui forme une liste chaînée de notes propre à chaque étudiant, permettant de stocker un nombre dynamique de modules sans utiliser de tableau statique.

```
1 typedef struct Etudiant
2 {
3     char nom[50];
4     char prenom[50];
5     int numero_dossier; // identifiant unique par etudiant
6     bool eta_inscription; // 1 si l etudiant s'est inscrit et
7     // 0 sinon (initialise lors de la creation)
8     float moyenne_general;
9     Note *modules;
10    struct Etudiant *etudiant_suivant; // permet de passer a
11    // l etudiant suivant
12 }Etudiant;
```

2.0.2 Explication de la structure Etudiant

La structure `Etudiant` regroupe l'ensemble des informations d'un étudiant de la filière, telles que son nom, son prénom, son numéro de dossier (identifiant unique) et son état d'inscription (via un booleen). Le pointeur `modules` est essentielle : il pointe vers la première note de l'étudiant, liant ainsi l'étudiant à ses résultats. Concernant le pointeur `etudiant_suivant` elle pointe sur l etudiant suivante creent ainsi une liste chaine

```
1 typedef struct
2 {
3     char nom_filiere[60];
4     int nb_etudiants;
5     Etudiant *premier_du_liste; // point sur le premier de la
6     // liste des etudiant
7     Etudiant *dernier_du_liste; // point sur le dernier
8     // etudiant de la liste
9 }Filiere;
```

2.0.3 Explication de la structure Filiere

Cette structure contient le nom d'une filiere donner ainsi que le nombre d'etudiants. Dans la structure `Filiere` on a deux pointeurs : `premier_du_liste` qui pointe sur le premier étudiant de la liste du filière, `dernier_du_liste` quant a lui pointe sur le dernier étudiant. Permettant d inserer des etudiants directemant a la fin de la liste

3 Modélisation des Fonctions

Voici les fonctions que nous avons conçue pour permettre le bon fonctionnement de notre Systeme de scolarité.

```
1 Etudiant *inscrire_etudiants(Filiere *filiere)
2 {
3     // allocation dynamique d un nouveau etudiant
4     Etudiant *nouveau_etudiant = (Etudiant*) malloc(sizeof(
        Etudiant));
5     if (nouveau_etudiant == NULL)
6     {
7         printf("Ereure: impossible d allouer la memoire");
8         return;
9     }
10    // ajout des information d un etudiant par l utilisateur
11    printf("\n-----Formulaire d inscription-----\n");
12    printf("Donner le nom de l etudiant: \n");
13    scanf("%s", nouveau_etudiant->nom);
14    printf("Donner le prenom de l etudiant: \n");
15    scanf("%s", nouveau_etudiant->prenom);
16    printf("Donner son numero de dossier: \n");
17    scanf("%d", &nouveau_etudiant->numero_dossier);
18
19    nouveau_etudiant->eta_inscription = 1;    // on va
        enregistrer l etat d inscription a 1 par default
20    nouveau_etudiant->modules = NULL ; // pas encore de note
21    nouveau_etudiant->etudiant_suivant = NULL; // se sera le
        dernier etudiant
22
23    if (filiere->premier_du_liste == NULL)
24    {
25        // La liste est vide, il est le tout premier et le
        tout dernier
26        filiere->premier_du_liste = nouveau_etudiant;
27        filiere->dernier_du_liste = nouveau_etudiant;
28    } else
29    {
30        filiere->dernier_du_liste->etudiant_suivant =
            nouveau_etudiant; // le pointeur dernier_du_liste
            pointe sure le nouveau etudiant
31        filiere->dernier_du_liste = nouveau_etudiant; // Il
            devient le nouveau dernier de la filiere
32    }
33
34    filiere->nb_etudiants++;
35    printf("L'etudiant %s %s a ete inscrit avec succes!\n",
        nouveau_etudiant->prenom, nouveau_etudiant->nom);
36    return nouveau_etudiant;
```

37 }

3.0.1 Explication de la Fonctions inscrire_etudiant

Cette fonction nous permet d'ajouter de nouveaux etudiant dans une filiere en enregistrant leur information personnel : prenom, nom, numero de dossier grace a l'allocation dynamique suivant : `Etudiant *nouveau_etudiant = (Etudiant*) malloc(sizeof(Etudiant))` qui alloue une espace memoire dans le memoire RAM. Combiner au boucle suivant.

```
1  if (filiere->premier_du_liste == NULL)
2  {
3      // La liste est vide, il est le tout premier et le
        tout dernier
4      filiere->premier_du_liste = nouveau_etudiant;
5      filiere->dernier_du_liste = nouveau_etudiant;
6  } else
7  {
8      filiere->dernier_du_liste->etudiant_suivant =
        nouveau_etudiant; // le pointeur dernier_du_liste
        pointe sure le nouveau etudiant
9      filiere->dernier_du_liste = nouveau_etudiant; // Il
        devient le nouveau dernier de la filiere
10 }
```

L'utilisateur pourra ajouter autant d'etudiant qu'il souhaite, le programme aura ainsi le nombre totale d'etudiants.

```
1 void note_etudiants(Etudiant *etudiant) // une fonction
    permettant d'enregistrer les resultat academique d'un
    etudiant
2 {
3     if (etudiant == NULL) return;
4
5     // On verifie si l'etudiant est bien inscrit
6     if (etudiant->eta_inscription == 0)
7     {
8         printf("Impossible d'ajouter une note : cet etudiant
            n'est pas activement inscrit.\n");
9         return;
10    }
11    int nouveaux_matiere = 2;
12    while (nouveaux_matiere == 2)
13    {
14        Note *nouveau_note = (Note*)malloc(sizeof(Note)); //
            allocation dynamique
15        if (nouveau_note == NULL)
16        {
```

```

17     printf("Erreur: impossible d'allouer la memoire");
18     return;
19 }
20
21
22 // ajout du resultat d un etudiant
23 printf("\n-----Resultats_Academique-----\n");
24 printf("Donner la matiere: \n");
25 scanf("%s", nouveau_note->matier);
26 do
27 {
28     printf("Donner la note obtenue: \n");
29     scanf("%f", &nouveau_note->valeur);
30     if (nouveau_note->valeur < 0 || nouveau_note->
        valeur > 20)
31     {
32         printf("Erreur! La note doit etre comprise
            entre 0 et 20.\n");
33     }
34 } while (nouveau_note->valeur < 0 || nouveau_note->
    valeur > 20);
35 do
36 {
37     printf("Donner le coefficient du matiere: \n");
38     scanf("%d", &nouveau_note->coef);
39     if (nouveau_note->coef <= 0 || nouveau_note->coef
        > 10)
40     {
41         printf("Erreur! Le coefficient doit etre
            compris entre 1 et 10.\n");
42     }
43 } while (nouveau_note->coef <= 0 || nouveau_note->
    coef > 10);
44
45 nouveau_note->note_suivante = NULL; // c est la
    premiere et dernier note
46
47 if (etudiant->modules == NULL)
48 {
49     // Cas ou l etudiant n a encore aucune note
50     etudiant->modules = nouveau_note;
51 }
52 else
53 {
54     // Cas ou l'etudiant a deja des notes, on
        parcourt pour aller a la fin
55     Note *note_restante = etudiant->modules;
56     while (note_restante->note_suivante != NULL)
57     {

```

```

58         note_restante = note_restante->note_suivante;
59     }
60     // On ajoute la nouvelle note tout a la fin
61     note_restante->note_suivante = nouveau_note;
62 }
63 printf("Matiere_ajouter_avec_succe:\n");
64 printf("Si_vous_vouler_ajouter_une_nouvelle_matiere_
    tape_2_si_non_0:\n");
65 scanf("%d", &nouveaux_matiere);
66 }
67 }

```

3.0.2 Explication de la Fonctions note_etudiant

Comme la fonction precedente celui ci Permet de stocke les resultats scolaire de chaque etudiants enregistrer en amont ,grace a l allocation dynamique suivant `Note *nouveau_note = (Note*)malloc(sizeof(Note))`. CEtte fonction va permettre a l utilisateur de donner autant de matiere ainsi que les note et coefficient associer. Pour chacune des etudiants qu il enregistre il peut aussi ne pas ajouter de note pour les etudiants qu il souhaite . Et cela grace au boucle `while (nouveaux_matiere == 2)` qui lui dira de taper 2 s il veut ajouter des matiere et 0 sinon .

```

1 void liste_de_la_classe(Filiere *filiere)
2 {
3     if (filiere == NULL || filiere->premier_du_liste == NULL)
4         // verifie si la filiere est vide ou non
5     {
6         printf("Aucun_etudiant_inscrit_dans_la_filiere.\n");
7         return;
8     }
9     printf("\n-----Liste_des_etudiants_de_la_classe
    -----\n");
10    Etudiant *actuel_liste = filiere->premier_du_liste;
11    while (actuel_liste != NULL) // affiche les identifiant
        des etudiants
12    {
13        printf("Nom:_%s,_Prenom:_%s,_Numero_de_dossier:_%d\n"
            ,actuel_liste->nom, actuel_liste->prenom,
            actuel_liste->numero_dossier);
14        actuel_liste = actuel_liste->etudiant_suivant;
15    }
16 }

```

3.0.3 Explication de la Fonctions liste_de_la_classe

Quant a cette fonction il affichera la liste des etudiants que l utilisateur a deja enregistrer .Il donne respectivement leur nom, prenom et numero de dossier.

```
1 void bulletin(Filiere *filiere)
2 {
3     if (filiere == NULL || filiere->premier_du_liste == NULL)
4     {
5         printf("Aucun_étudiant_inscrit_dans_la_filiere.\n");
6         return;
7     }
8
9     printf("\n-----Bulletin_des_étudiants_de_la_classe
10         -----\n");
11
12     float somme_des_moyennes_classe = 0;
13     int etudiants_avec_notes = 0; // prend en compte que les
14                                     etudiants ayant une moyenne
15
16     Etudiant *actuel_liste_info = filiere->premier_du_liste;
17
18     while (actuel_liste_info != NULL)
19     {
20         // On verifie le statut d inscription de chaque
21         // etudiant
22         if (actuel_liste_info->eta_inscription == 1)
23         {
24             printf("Nom:_%s,_Prenom:_%s,_Numero_de_dossier:_%d\n",
25                 actuel_liste_info->nom,
26                 actuel_liste_info->prenom, actuel_liste_info->
27                 numero_dossier);
28
29             Note *actuel_liste_bulletin = actuel_liste_info->
30                 modules;
31             float point_totale = 0;
32             int coef_totale = 0;
33
34             while (actuel_liste_bulletin != NULL)
35             {
36                 printf("Matiere:_%s,_Note:_%f,_Coeficiant:_%d\n",
37                     actuel_liste_bulletin->matier,
38                     actuel_liste_bulletin->valeur,
39                     actuel_liste_bulletin->coef);
40
41                 point_totale += actuel_liste_bulletin->valeur
42                     * actuel_liste_bulletin->coef;
43                 coef_totale += actuel_liste_bulletin->coef;
44                 actuel_liste_bulletin = actuel_liste_bulletin
45                     ->note_suivante;
46             }
47         }
48     }
49 }
```

```

32         }
33
34         if (coef_totale != 0)
35         {
36             actuel_liste_info->moyenne_general =
37                 point_totale / coef_totale;
38             printf("Moyenne_Generale: %.2f\n\n",
39                 actuel_liste_info->moyenne_general);
40             somme_des_moyennes_classe +=
41                 actuel_liste_info->moyenne_general;
42             etudiants_avec_notes++;
43         }
44         else
45         {
46             actuel_liste_info->moyenne_general = 0;
47             printf("Aucune_note_enregistre\n");
48         }
49     }
50
51     // On passe a l etudiant suivant
52     actuel_liste_info = actuel_liste_info->
53         etudiant_suivant;
54
55 }
56
57 // Affichage de la moyenne gennerale
58 if (etudiants_avec_notes != 0)
59 {
60     float moyenne_general_classe =
61         somme_des_moyennes_classe / etudiants_avec_notes;
62     printf("MOYENNE_GENERALE_DE_LA_FILIERE: %.2f / 20\n",
63         , moyenne_general_classe);
64 }
65 else
66 {
67     printf("Les etudiants non pas de note.\n");
68 }
69 }
70 // fonction pour rechercher un etudiant

```

3.0.4 Explication de la Fonctions bulletin

La fonction bulletin permet de calculer les moyenne de chaque etudiant ainsi que la moyenne gennerale de la classe. Elle va aller chercher les module de chaque etudiants c est a dire ces matiere ainsi que les coef et notes associer pour calculer sa moyenne. Si l etudiant n a pas de note la boucle :

```

1         {
2             actuel_liste_info->moyenne_general = 0;
3             printf("Aucune_note_enregistre\n");

```



```

4         }
5     }

```

Va afficher a l ecran que cette etudiant n a pas de note ,cette etudiant ne sera pas prise en compte pour le calcule du moyenne gennerale de la classe . Grace au variable `etudiants_avec_notes` , innicialise a 0 qui n avance que si l etudiant qu on doit calculer la moyenne a une note .De cette facon avec la boucle :

```

1     {
2         float moyenne_general_classe =
            somme_des_moyennes_classe / etudiants_avec_notes;
3         printf("MOYENNE_GENERALE_DE_LA_FILIERE: %.2f / 20\n"
            , moyenne_general_classe);
4     }
5     else
6     {
7         printf("Les etudiants non pas de note.\n");
8     }
9 }
10 // fonction pour rechercher un etudiant

```

Cqlcule et affiche la moyenne gennerale exacte de la classe.

```

1 {
2     int votre_choix = 3;
3     int choix;
4     char matier[50];
5     float note;
6     int coef;
7     while (votre_choix == 3)
8     {
9         printf("Voulez vous rechercher ou modifier les info d
            un autre etudiant si oui taper 3 si non 0:\n");
10        scanf("%d", &votre_choix);
11        if (votre_choix == 0)
12        {
13            break;
14        }
15        if (filiere == NULL || filiere->premier_du_liste ==
            NULL)
16        {
17            printf("Aucun etudiant dans la filiere.\n");
18            return;
19        }
20        int id_rechercher;
21        printf("Donner le numero de dossier de l etudiant
            rechercher:\n");
22        scanf("%d", &id_rechercher);
23

```

```

24     Etudiant *etudiant_actuel = filiere->premier_du_liste
      ;
25     while (etudiant_actuel != NULL && etudiant_actuel->
      numero_dossier != id_rechercher)
26     {
27         etudiant_actuel = etudiant_actuel->
            etudiant_suivant;
28     }
29     if (etudiant_actuel == NULL)
30     {
31         printf("%d ne correspond a aucun etudiant
            enregistre:\n", id_rechercher);
32     }
33     else
34     {
35         printf("\n-----Etudiant trouver-----\n");
36         printf("Nom: %s, Prenom: %s, Numero de dossier: %d,
            Etat d'inscription: %s", etudiant_actuel->
            nom, etudiant_actuel->prenom, etudiant_actuel
            ->numero_dossier, (etudiant_actuel->
            eta_inscription == 1) ? "Inscrit" : "Non
            inscrit");
37         printf("\n-----Que souhaitez-vous modifier
            -----\n");
38         printf("1 Le Nom\n");
39         printf("2 Le Prenom\n");
40         printf("3 Le Numero de dossier\n");
41         printf("4 L'etat d'inscription (1: inscrit, 0:
            Suspendu)\n");
42         printf("5 Matiere Note et Coef\n");
43         printf("Votre choix: ");
44         scanf("%d", &choix);
45     }
46     switch (choix) // cette partie va permettre a l
        utilisateur de modifier les parametre qu il
        souhaite
47     {
48         case 1:
49             printf("Entrez le nouveau nom: \n");
50             scanf("%s", etudiant_actuel->nom);
51             printf("Nom modifie avec succes!\n");
52             break;
53         case 2:
54             printf("Entrez le nouveau prenom: \n");
55             scanf("%s", etudiant_actuel->prenom);
56             printf("Prenom modifie avec succes!\n");
57             break;
58         case 3:
59             printf("Entrez le nouveau numero de dossier

```

```

        :\n");
60     scanf("%d", &etudiant_actuel->numero_dossier
        );
61     printf("Numero de dossier modifie avec succes
        \n");
62     break;
63     case 4:
64     printf("Entrez le nouveau statut (1 pour
        Inscrit, 0 pour Non inscrit):\n");
65     scanf("%d", &etudiant_actuel->
        eta_inscription);
66     printf("Statut d'inscription mis a jour!\n")
        ;
67     break;
68     case 5:
69     if (etudiant_actuel->modules == NULL)
70     {
71         etudiant_actuel->modules = malloc(sizeof*(
        etudiant_actuel->modules));
72         if (etudiant_actuel->modules == NULL)
73         {
74             printf("Ereur de l'allocation dynamique
        ");
75         }
76     }
77     Note *etudiant_actuel_modules = NULL;
78     Note *actuel = etudiant_actuel->modules;
79     while (actuel->note_suivante != NULL)
80     {
81         actuel = actuel->note_suivante;
82     }

```

3.0.5 Explication de la Fonctions recherche_et_modification_etudiant

Cette fonction va demander a l'utilisateur s'il veut rechercher un etudiant afin de modifier ou non ces informations personnelles ou académiques en lui affichant un message lui permettant de faire deux choix soit taper 3 pour continuer et avoir la possibilité de modifier certaines informations. Soit taper 0 pour arrêter le programme, et éventuellement ne rien modifier ou sortir après des modifications.

```

1
2     printf("Entrez la nouvelle matiere:\n");
3     scanf("%s", matier);
4     if (strcmp(etudiant_actuel_modules->matier,
        matier) == 0 )
5     {
6         printf("Entrez la nouvelle note:\n");

```

```

7         scanf("%f", &etudiant_actuel_modules->valeur)
            ;
8         printf("Entrez le nouveau coef:\n");
9         scanf("%d", &etudiant_actuel_modules->coef);
10        printf("La matiere ainsi que sa note et son
            coef on ete modifier avec succe!\n");
11
12    }
13    else
14    {
15        etudiant_actuel_modules->note_suivante =
            malloc(sizeof(Note));
16        if (etudiant_actuel_modules->note_suivante ==
            NULL)
17        {
18            printf("Erreur d'allocation memoire\n");
19            break;
20        }
21        strcpy(etudiant_actuel_modules->note_suivante
            ->matier, matier);
22        printf("Donner la note de la nouvelle matier
            :\n");
23        scanf("%f", &note);
24        etudiant_actuel_modules->note_suivante->
            valeur = note;
25        printf("Donner le coef de la nouvelle matier
            :\n");
26        scanf("%d", &coef);
27        etudiant_actuel_modules->note_suivante->coef
            = coef;
28        etudiant_actuel_modules->note_suivante->
            note_suivante = NULL;
29    }
30    break;
31    default:
32        printf("Choix invalide. Aucune modification
            effectuee.\n");
33        break;
34    }
35 }
36 }
37 // fonction pour supprimer un etudiant de la liste
38 void supprimer_etudiant(Filiere *filiere)
39 {
40     int votre_choix = 4;
41     while (votre_choix == 4)
42     {
43         printf("Si vous voulez supprimer une autre etudiant
            tape 4 sinon 0:\n");

```

```

44     scanf("%d", &votre_choix);
45     if (votre_choix == 0)
46     {
47         break;
48     }
49
50     if (filiere == NULL || filiere->premier_du_liste ==
        NULL)
51     {
52         printf("Il n'y a aucune etudiant");
53         return;
54     }
55     int id_supprimer;
56     printf("Donner le numero de dossier de l etudiant que
        vous souhaitez supprimer de la filiere:\n");
57     scanf("%d", &id_supprimer);
58     Etudiant *actuel_etudiant = filiere->premier_du_liste
        ;
59     Etudiant *precedent_etudiant = NULL;
60     while (actuel_etudiant != NULL && id_supprimer !=
        actuel_etudiant->numero_dossier)

```

3.0.6 Explication de la Fonctions supprimer_etudiant

De la meme manier que la fonction precedente celui ci lui permetra de supprimer ou non un etudiant de la liste

```

1         precedent_etudiant = actuel_etudiant; // 1
           etudiant precedent devient l actuel etudiant
2         actuel_etudiant = actuel_etudiant->
           etudiant_suivant; // 1 actuel etudiant pas a l
           etudiant suivant
3     }
4     if (actuel_etudiant == NULL)
5     {
6         printf("Etudiant introuvable avec l id: %d\n",
           id_supprimer);
7     }
8     else
9     {
10        if (precedent_etudiant == NULL)
11        {
12            // correspond au premier etudiant
13            filiere->premier_du_liste = actuel_etudiant->
                etudiant_suivant;
14            // cas ou la liste contien q un seul etudiant
15            if (filiere->premier_du_liste == NULL)
16            {

```

```

17         filiere->dernier_du_liste = NULL;
18     }
19
20 }
21 else
22 {
23     // 1 etudiant est au milieux ou a la fin
24     precedent_etudiant->etudiant_suivant =
        actuel_etudiant->etudiant_suivant;
25     if (actuel_etudiant == filiere->
        dernier_du_liste)
26     {
27         filiere->dernier_du_liste =
            precedent_etudiant;

```

3.0.7 Explication de la Fonctions maximum

Cette fonction va chercher parmit tout les etudiant celui qui a la plus grande moyenne et a la fin du programme il donne sont nom prenom numero de dossier sa moyenne et le nom de la filiere.

```

1     }
2     free(actuel_etudiant); // liber l espace allouher
        a l etudiant supprimer
3     filiere->nb_etudiants--;
4     printf("L'etudiant avec l'id:%d a ete supprimer
        avec succe\n", id_supprimer);
5 }
6 }
7 }
8 // fonction pour trouver le majorant de la filiere
9 void maximum(Filiere *filiere)
10 {
11     if (filiere == NULL || filiere->premier_du_liste == NULL)
12     {
13         printf("Il n'y a aucune etudiant");
14         return;
15     }
16     float majorant = 0.0;
17     Etudiant *etudiant_major = filiere->premier_du_liste;
18     Etudiant *le_major = NULL; // Pointeur pour memoriser le
        majorant
19     while (etudiant_major != NULL)
20     {
21         if (etudiant_major->eta_inscription == 1 &&
            etudiant_major->moyenne_general >= 0.0)
22         {
23             if (etudiant_major->moyenne_general > majorant)

```

```

24         {
25             majorant = etudiant_major->moyenne_general;
                // mise a jour du nouveaux majorant
26             le_major = etudiant_major; // memorise l
                enplacement du majorant actuelle
27         }

```

3.0.8 Explication de la Fonctions minimum

Pareil pour la fonction minimum ,il donne l etudiant ayant la plus faible moyenne de la filiere .

```

1         etudiant_major = etudiant_major->etudiant_suivant;
2     }
3     if (le_major != NULL)
4     {
5         printf("Le_majorant_de_la_filiere:%s,est%s,%s,
                numero_de_dossier:%d,et_a_une_moyenne_generale
                de:%.2f\n",filiere->nom_filiere, le_major->prenom
                , le_major->nom, le_major->numero_dossier,
                le_major->moyenne_general);
6     }
7 }
8 // fonction pour trouver le minorant
9 void minimum(Filiere *filiere)
10 {
11     if (filiere == NULL || filiere->premier_du_liste == NULL)
12     {
13         printf("Il_n_y_a_aucune_etudiant");
14         return;
15     }
16     float minorant = 20;
17     Etudiant *etudiant_minor = filiere->premier_du_liste;
18     Etudiant *le_minor = NULL; // Pointeur pour memoriser le
        minorant
19     while (etudiant_minor != NULL)
20     {
21         if (etudiant_minor->eta_inscription == 1 &&
            etudiant_minor->moyenne_general >= 0.0)
22         {
23             if (etudiant_minor->moyenne_general < minorant)
24             {
25                 minorant = etudiant_minor->moyenne_general;
                // mise a jour du nouveaux minorant
26                 le_minor = etudiant_minor; // memorise l
                enplacement du minorant actuelle
27             }
28         }

```

```

29     etudiant_minor = etudiant_minor->etudiant_suivant;
30 }
31 if (le_minor != NULL)
32 {
33     printf("L'etudiant ayant la plus faible note de la
        filiere:%s, est %s, %s, numero de dossier: %d,
        et a une moyenne generale de: %.2f\n", filiere->
        nom_filiere, le_minor->prenom, le_minor->nom,
        le_minor->numero_dossier, le_minor->
        moyenne_general);
34 }
35 }
36 // tri par ordre de merite (tri par selection)
37 void du_majorant_minorant(Filiere *filiere)
38 {
39     if (filiere == NULL || filiere->premier_du_liste == NULL)
40     {
41         printf("Il n'y a aucune etudiant");
42         return;
43     }
44     Etudiant *i = filiere->premier_du_liste;
45     Etudiant *j;
46     Etudiant *major_minor = NULL; // Pointeur pour memoriser
        le i
47     // il nous faut des variable et une pointeur pour les
        echange fineaux
48     char tmp_nom[50];
49     char tmp_prenom[50];
50     int tmp_id;
51     int tmp_etat_inscription;
52     float tmp_moyenne_general;
53     Note *tmp_modules;
54     while (i != NULL)
55     {

```

3.0.9 Explication de la Fonctions du_majorant_minorant

Cette fonction Permet d'afficher par ordre de merite tout les etudiant de la filiere du majorant a l'etudiant ayant la plus faible moyenne grace a la trie par selection :

```

1     Etudiant *le_minor = NULL; // Pointeur pour memoriser le
        minorant
2     while (etudiant_minor != NULL)
3     {
4         if (etudiant_minor->eta_inscription == 1 &&
            etudiant_minor->moyenne_general >= 0.0)
5         {

```



```

6         if (etudiant_minor->moyenne_general < minorant)
7         {
8             minorant = etudiant_minor->moyenne_general;
9             // mise a jour du nouveaux minorant
10            le_minor = etudiant_minor; // memorise l
11            // emplacement du minorant actuelle
12        }
13    }
14    etudiant_minor = etudiant_minor->etudiant_suivant;
15 }
16 if (le_minor != NULL)
17 {
18     printf("L'etudiant ayant la plus faible note de la
19     filiere: %s, est %s, %s, numero de dossier: %d,
20     et une moyenne generale de: %.2f\n", filiere->
21     nom_filiere, le_minor->prenom, le_minor->nom,
22     le_minor->numero_dossier, le_minor->
23     moyenne_general);
24 }
25 }
26 // tri par ordre de merite (tri par selection)
27 void du_majorant_minorant(Filiere *filiere)
28 {
29     if (filiere == NULL || filiere->premier_du_liste == NULL)
30     {
31         printf("Il n'y a aucune etudiant");
32         return;
33     }
34     Etudiant *i = filiere->premier_du_liste;
35     Etudiant *j;
36     Etudiant *major_minor = NULL; // Pointeur pour memoriser
37     le i
38     // il nous faut des variable et une pointeur pour les
39     echage fineaux
40     char tmp_nom[50];
41     char tmp_prenom[50];
42     int tmp_id;
43     int tmp_etat_inscription;
44     float tmp_moyenne_general;
45     Note *tmp_modules;
46     while (i != NULL)
47     {
48         j = i->etudiant_suivant;
49         while (j != NULL)
50         {
51             if (j->moyenne_general > major_minor->
52             moyenne_general)
53             {
54                 major_minor = j; // j devient temporairement

```

```

                                le plus grand
7         }
8         j = j->etudiant_suivant; // j passe a l etudiant
                                suivant
9     }
10    if (major_minor != i)
11    {
12        strcpy(tmp_nom, i->nom);
13        strcpy(tmp_prenom, i->prenom);
14        tmp_id = i->numero_dossier;
15        tmp_etat_inscription = i->eta_inscription;
16        tmp_moyenne_general = i->moyenne_general;
17        tmp_modules = i->modules;
18
19        strcpy(i->nom, major_minor->nom);
20        strcpy(i->prenom, major_minor->prenom);
21        i->numero_dossier = major_minor->numero_dossier;
22        i->eta_inscription = major_minor->eta_inscription
23        ;
24        i->moyenne_general = major_minor->moyenne_general
25        ;
26        i->modules = major_minor->modules;
27
28        strcpy(major_minor->nom, tmp_nom);
29        strcpy(major_minor->prenom, tmp_prenom);
30        major_minor->numero_dossier = tmp_id;
31        major_minor->eta_inscription =
32            tmp_etat_inscription;
33        major_minor->moyenne_general =
34            tmp_moyenne_general;
35        major_minor->modules = tmp_modules;
36    }
37    i = i->etudiant_suivant;
38 }
39 }
40 // tri par ordre alphabetique (tri a bulle)
41 void orde_alphabet(Filiere *filiere)
42 {
43     if (filiere == NULL || filiere->premier_du_liste == NULL)
44     {
45         printf("Il n'y a aucune etudiant\n");
46         return;
47     }
48     Etudiant *i = filiere->premier_du_liste;
49     Etudiant *j;
50
51     char tmp_nom[50];
52     char tmp_prenom[50];

```

```

50     int tmp_id;
51     int tmp_etat_inscription;
52     float tmp_moyenne_general;
53     Note *tmp_modules;

```

3.0.10 Explication de la Fonctions orde_alphabet

Conservant cette fonction elle utilise une trie a bulle, et elle va permettre l'affichage de la liste par ordre alphabetique :

```

1         strcpy(i->nom, major_minor->nom);
2         strcpy(i->prenom, major_minor->prenom);
3         i->numero_dossier = major_minor->numero_dossier;
4         i->eta_inscription = major_minor->eta_inscription
5         ;
6         i->moyenne_general = major_minor->moyenne_general
7         ;
8         i->modules = major_minor->modules;
9
10        strcpy(major_minor->nom, tmp_nom);
11        strcpy(major_minor->prenom, tmp_prenom);
12        major_minor->numero_dossier = tmp_id;
13        major_minor->eta_inscription =
14        tmp_etat_inscription;
15        major_minor->moyenne_general =
16        tmp_moyenne_general;
17        major_minor->modules = tmp_modules;
18    }
19    i = i->etudiant_suivant;
20 }
21 }
22 // tri par ordre alphabetique (tri a bulle)
23 void orde_alphabet(Filiere *filiere)
24 {
25     if (filiere == NULL || filiere->premier_du_liste == NULL)
26     {
27         printf("Il n'y a aucune etudiant\n");
28         return;
29     }
30
31     Etudiant *i = filiere->premier_du_liste;
32     Etudiant *j;
33
34     char tmp_nom[50];
35     char tmp_prenom[50];
36     int tmp_id;
37     int tmp_etat_inscription;
38     float tmp_moyenne_general;

```

4 Modélisation du corps de notre projet : (int main)

C'est ici que nous faisons les appels des différentes fonctions pour donner vie à notre programme.

```
1 int main()
2 {
3     int votre_choix1 = 1;
4     int votre_choix2 = 1;
5     Etudiant *nouveau_etudiant = NULL;
6     Filiere ma_filiere;
7     ma_filiere.premier_du_liste = NULL;
8     ma_filiere.dernier_du_liste = NULL;
9     ma_filiere.nb_etudiants = 0;
10
11     printf("=====\u00a0BIENVENUE!\u00a0=====\n");
12     printf("Veuillez\u00a0donner\u00a0le\u00a0nom\u00a0de\u00a0votre\u00a0filiere:\n");
13     scanf("%s", ma_filiere.nom_filiere);
14     while (votre_choix1 != 0)
15     {
16         nouveau_etudiant = inscrire_etudiants(&ma_filiere);
17         printf("Voulez\u00a0vous\u00a0ajouter\u00a0ces\u00a0note\u00a0si\u00a0oui\u00a0tape\u00a0(1)\u00a0\u00a0si\u00a0non\u00a0(0)\n");
18         scanf("%d", &votre_choix2);
19         if (votre_choix2 == 1)
20         {
21             note_etudiants(nouveau_etudiant);
22         }
23         printf("Voulez\u00a0vous\u00a0inscrire\u00a0un\u00a0nouveau\u00a0etudiant\u00a0si\u00a0oui\u00a0tape\u00a0(1)\u00a0si\u00a0non\u00a0(0)\n");
24         scanf("%d", &votre_choix1);
25     }
26     liste_de_la_classe(&ma_filiere);
27     bulletin(&ma_filiere);
28     recherche_et_modification_etudiant(&ma_filiere);
29     supprimer_etudiant(&ma_filiere);
30     printf("=====\u00a0Bulletin\u00a0apres\u00a0modification!\u00a0=====");
31     bulletin(&ma_filiere);
32     maximum(&ma_filiere);
33     minimum(&ma_filiere);
34     du_majorant_minorant(&ma_filiere);
35     printf("-----\u00a0Classe\u00a0triee\u00a0par\u00a0ordre\u00a0de\u00a0merite\u00a0du\u00a0meilleure\u00a0au\u00a0moins\u00a0bon\u00a0-----\n");
36     liste_de_la_classe(&ma_filiere);
37     orde_alphabet(&ma_filiere);
38     printf("-----\u00a0Classe\u00a0triee\u00a0par\u00a0ordre\u00a0alphabetique\u00a0-----\n");
```

```
39     liste_de_la_classe(&ma_filiere);
```

5 Etude